Final Project Presentation on

Real-Time Dynamic Path Planning via Learned First-Order Perturbations of Laplacian Potential Fields

Boxiang (William) Fu

16-832 (Spring 2026) Integrated Planning and Learning
Carnegie Mellon University

04/20/2026

Motivation

Two equally valid descriptions of the universe...

Newtonian Mechanics

- Local approach
- Mediated by Force particles
- Solves for cause-and-effect one step at a time

Lagrangian Mechanics

- Global approach
- Mediated by Force fields
- Solves for entire trajectory using the Principle of Least Action

Robotics Application

- Dijkstra's and A* algorithms
- Graph traversal algorithms

Robotics Application

- Navigation functions
- Laplacian potential fields

Problem Statement

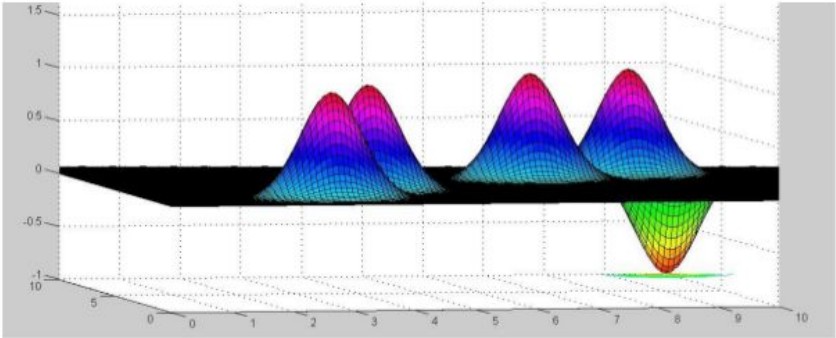
- Given an initial static map with unknown dynamic obstacles, solve the entire (global) trajectory in real-time to the goal location
- Use case: Shopping malls with known floor layout and dynamic pedestrians. Robot must reach goal location while avoiding pedestrians

Related Work

- Path planning via navigation functions (Rimon & Koditschek, 1988)
- Laplace's equation as navigation function (Connolly et al., 1990)
- Heat transfer equation as navigation function (Wang and Chirikjian, 2000)
- Path planning via Hamiltonians (Casagrande et al., 2017)
- Artificial potential fields (Wu et al., 2023)
- Poisson potential fields for guiding MAPF/TAPF (My 16-891 Project)
- Dynamic path planning via perturbed Laplace's equation (Ours)

Methodology (Static Map)

- Model static obstacles and map boundaries as Neumann boundary conditions (i.e. normal derivative is 0)
- Model source location as positively charged source particle
- Model goal location as negatively charged sink particle
- Solve for Laplace's equation in free-space ($\nabla^2\varphi(x) = 0$)
- Robot will follow the path of least resistance ($dx/dt = -\nabla\varphi(x)$)

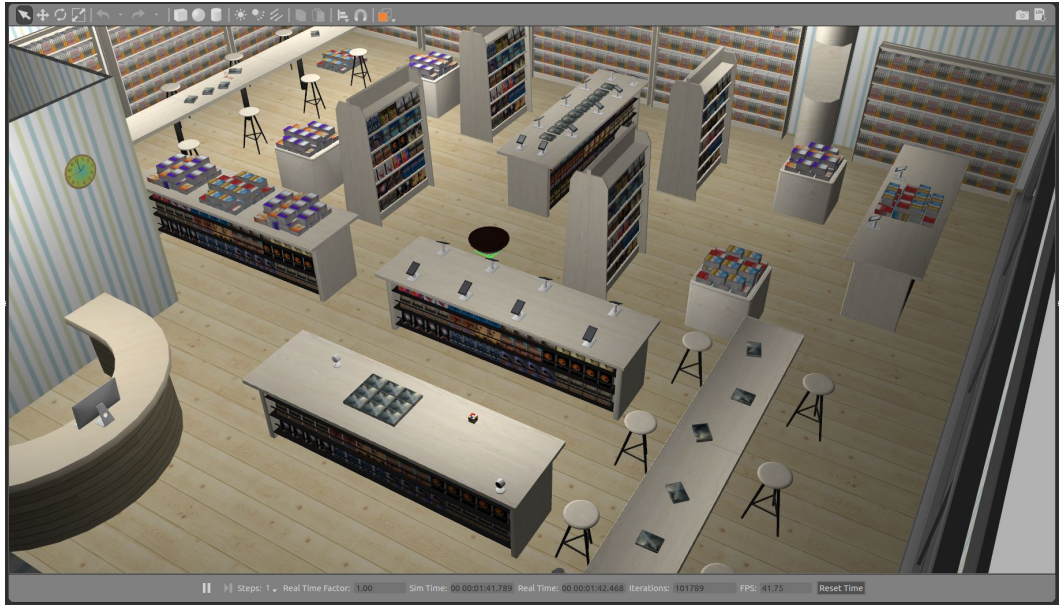


Methodology (Dynamic Map)

- Look for disturbances in the local neighborhood of the robot
- Model additional dynamic obstacles as Neumann boundary conditions
- Impose Dirichlet conditions equal to the original potential field at the boundary of local neighborhood
- Use successive over-relaxation or neural network to obtain first-order perturbation in the potential field due to the new particles
- Superimpose the first-order perturbation with the original potential field
- Robot will follow the (new) path of least resistance ($dx/dt = -\nabla\varphi(x)$)
- Repeat for next timestep

Setup – Simulation

- Robot modelled as TurtleBot Burger with 360 degree FOV within a 2 meter radius
- Bookstore modelled by extending Bookstore_World in Gazebo simulator
- 3 humans added to environment at locations not reflected in static map



Setup – Pathfinder

- Implemented in ROS2 primarily using 4 nodes:
- **Costmap** (from nav2): Maintains a real-time local occupancy costmap from sensor data and feeds dynamic obstacle information to the perturbation node
- **Perturbation**: Detects dynamic obstacles via the local costmap, solves a local Laplace patch to compute a perturbation field, and publishes it for the planner
- **Robot_nav_planner**: Gradient descent waypoint planner over a precomputed Laplace potential field with online perturbation for dynamic obstacles
- **Robot_nav_controller**: Executes low-level motion control using proportional control for linear and angular movement

Pseudocode – Perturbation

Algorithm 1 Perturbation Node

```
Load global map and potential field φ
On local costmap update:
if all local obstacles exist in global map then
    Publish zero perturbation field Δφ = 0
    return
end if
Extract local patch φ_patch
Overlay new obstacles and inflate them
Mark obstacles and boundaries as fixed (φ = 0)
Solve Laplace on patch:
if solver = SOR then
    Run SOR iterations on φ_patch
    Δφ ← φ_patch − φ_global
else if solver = NN + SOR then
    Predict Δφ with NN, refine with SOR
else if solver = NN only then
    Predict Δφ with NN
end if
Publish Δφ
```

Pseudocode – Robot_nav_planner

Algorithm 2 Robot_nav_planner Node

```
Load potential field φ
Initialize Δφ ← 0
Set goal threshold φ_goal
On each timer tick:
(r, c) ← current grid cell
if φ[r, c] ≥ φ_goal then
    return
end if
for i = 1 to lookahead do
    (r, c) ← arg max_{(n_r, n_c) ∈ N(r,c)} (φ + Δφ)
end for
Publish waypoint from (r, c)
On perturbation update:
    Update Δφ
```

Pseudocode – Robot_nav_controller

Algorithm 3 Robot_nav_controller Node

```
Set gains k_lin, k_ang and tolerance ε
On waypoint update:
    Set goal (x_g, y_g), activate goal
On odometry update:
if no active goal then
    Publish zero velocity
    return
end if
Compute distance d ← ||(x, y) − (x_g, y_g)||
if d < ε then
    Clear goal, publish zero velocity
    return
end if
Compute heading error θ_e
ω ← k_ang · θ_e
v ← k_lin · d · max(0, cos(θ_e))
Publish (v, ω)
```

Theoretical Properties – Lemma 1

Lemma 1 (Field Lines Connect Source to Sink). *Let $\Omega \subset \mathbb{R}^2$ be a bounded, connected open set with boundary $\partial\Omega$, and impose Neumann boundary conditions on $\partial\Omega$.*

Let $\rho \in L^1(\Omega)$ be a charge distribution such that:

$$\rho(x) > 0 \text{ in a source region}, \quad \rho(x) < 0 \text{ in a sink region}, \quad \text{and} \quad \int_\Omega \rho(x)\,dx = 0.$$

Let ϕ solve Poisson's equation:

$$\nabla^2\phi = -\rho \quad \text{in } \Omega,$$

with Neumann boundary conditions:

$$\nabla\phi \cdot n = 0 \quad \text{on } \partial\Omega.$$

Define the vector field:

$$\mathbf{E} = -\nabla\phi.$$

Then, the integral curves (field lines) of $\mathbf{E}$:

- *originate at source regions ($\rho > 0$),*
- *terminate at sink regions ($\rho < 0$),*
- *and do not form closed loops or terminate in free space.*

Theoretical Properties – Lemma 1

Proof Sketch. Consider the possible behaviors of field lines:

1. **Field lines cannot stop in free space.** If a field line terminated at a point where $\rho = 0$, then this would imply a local extrema of ϕ in a region where $\nabla^2\phi = 0$, which contradicts the maximum principle.

2. **Field lines cannot form closed loops.** Since $\mathbf{E} = -\nabla\phi$ is a gradient field, it is curl-free:

$$\nabla \times \mathbf{E} = 0.$$

   Therefore, line integrals are path-independent, and closed loops are impossible.

3. **Field lines must start/end at sources or sinks.** From Gauss's law:

$$\nabla \cdot \mathbf{E} = \rho,$$

   so divergence is nonzero only at sources and sinks. Thus, field lines can only originate where $\rho > 0$ and terminate where $\rho < 0$.

4. **Boundary behavior.** The Neumann condition $\nabla\phi \cdot n = 0$ implies $\mathbf{E}$ is tangent to the boundary, so field lines neither enter nor leave Ω through $\partial\Omega$.

Combining these facts, the only possible configuration is that field lines connect sources to sinks. □

Theoretical Properties – Lemma 2

Lemma 2 (Boundary-to-boundary field lines in a bounded source-free region)**.** *Let $\Omega_L \subset \Omega$ be a bounded, connected, source-free region. Let ϕ satisfy*

$$\nabla^2 \phi = 0 \qquad \text{in } \Omega_L,$$

with mixed boundary conditions on $\partial\Omega_L$ inherited from the global problem, with boundary values fixed by the solved potential. Then the field lines of

$$\mathbf{F} = -\nabla\phi$$

start and end on the boundary $\partial\Omega_L$.

Proof Sketch. By the maximum principle, a harmonic function has no interior maxima or minima unless it is constant. Since field lines follow $-\nabla\phi$, they cannot originate or terminate at an interior point of Ω_L. Therefore, in a bounded connected source-free region, field lines must connect boundary points. □

Theoretical Properties – Theorem

Theorem 3 (No trapping under harmonic field following)**.** *Let $\Omega \subset \mathbb{R}^n$ be a connected domain containing a source and a sink. Let $\phi \in C^2(\Omega \setminus \{x_s, x_t\}) \cap C^0(\Omega)$ be the potential obtained by solving*

$$\nabla^2 \phi = -\rho \quad \text{in } \Omega,$$

where $\rho = \delta(x - x_s) - \delta(x - x_t)$ encodes a source at x_s and a sink at x_t, together with appropriate boundary conditions on $\partial\Omega$.

Define the vector field

$$\mathbf{F} = -\nabla \phi.$$

Let $\Omega_L \subset \Omega$ be a bounded, connected, source-free subregion. Assume any dynamic obstacles contained in Ω_L are fully enclosed, do not partition Ω_L, and are incorporated through boundary conditions on $\partial\Omega_L$.

Then an agent following the field lines of $\mathbf{F}$ moves from the source to the sink and does not get stuck.

Theoretical Properties – Theorem

Proof Sketch. Let a be any point on a field line followed by the agent.

Case 1: $a \notin \Omega_L$**.** Then a lies in the global free-space region. By Lemma 1, field lines of $\mathbf{F}$ connect the source to the sink. Hence, the agent continues along a trajectory toward the sink.

Case 2: $a \in \Omega_L$**.** Then a lies in a bounded, connected, source-free region. By Lemma 2, field lines cannot originate or terminate in the interior of Ω_L, and must intersect the boundary $\partial\Omega_L$. Thus, the agent cannot get stuck inside Ω_L and must exit the region.

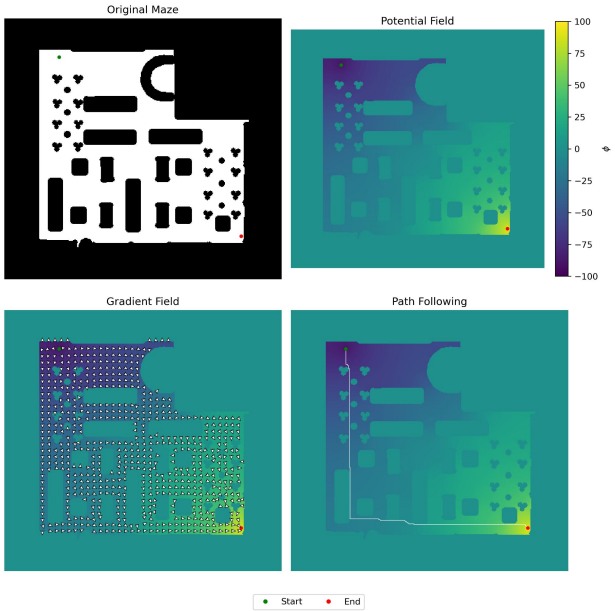
Since the dynamic obstacles do not partition Ω_L, the field line remains connected to the surrounding free space after reaching $\partial\Omega_L$. Once outside Ω_L, Lemma 1 applies again, and the agent continues toward the sink.

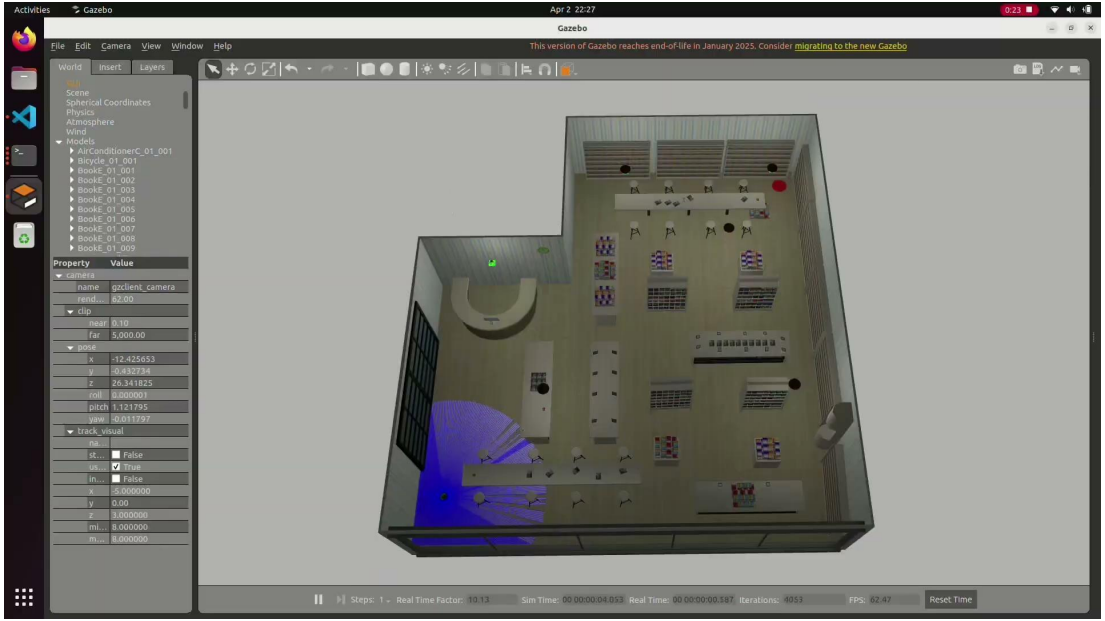
Therefore, for any point a along the trajectory, the agent progresses along a field line from the source to the sink and cannot get stuck. $\square$

Theoretical Properties – Breaking Assumptions

- Discretized map is not continuous and analytic
  - Not a huge problem, static pathfinding still performs well
- Dynamic obstacles are not fully observable
  - Lidar scan cannot see the “back-side” of the obstacle
  - Dynamic obstacles may not be fully enclosed on the local region
  - Some field lines route into the unobserved “back-side” of the obstacle, causing collisions
- Robot and obstacles are not point masses
  - Their presence could sometimes partition the local neighborhood, rendering the solver unable to find a solution
- Some caveats and border cases of the Lemmas and Theorem still yet to be worked out…

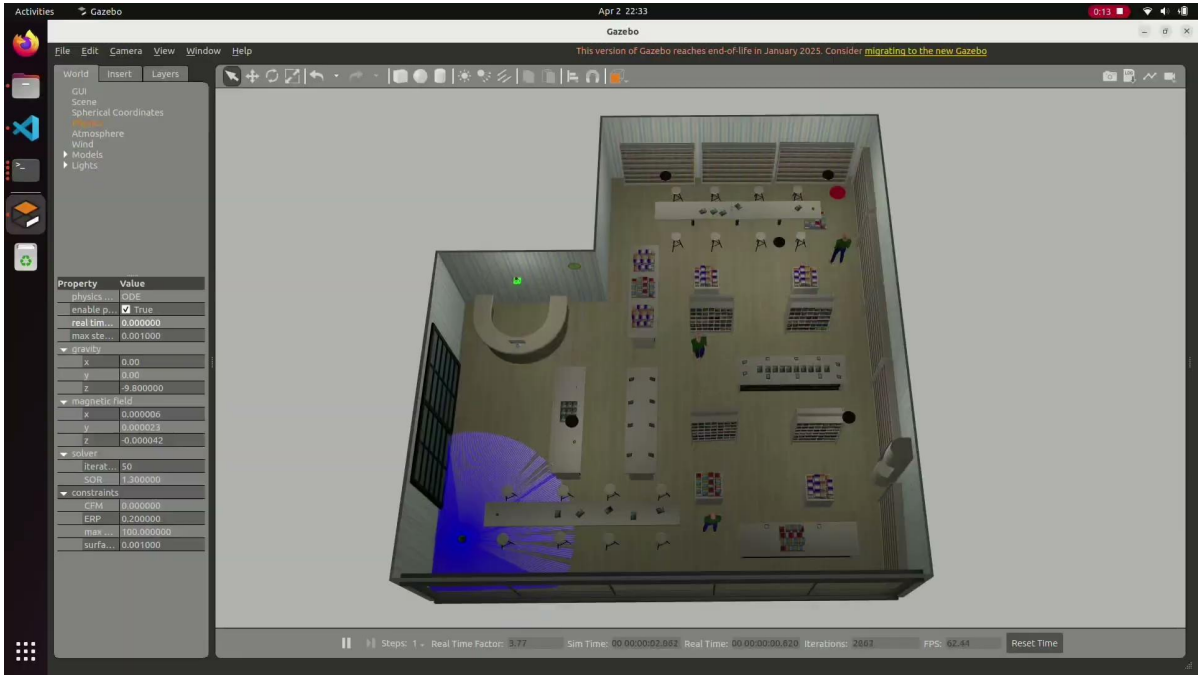
Results – No Dynamic Obstacles





- Static pathfinder using potential field works as intended

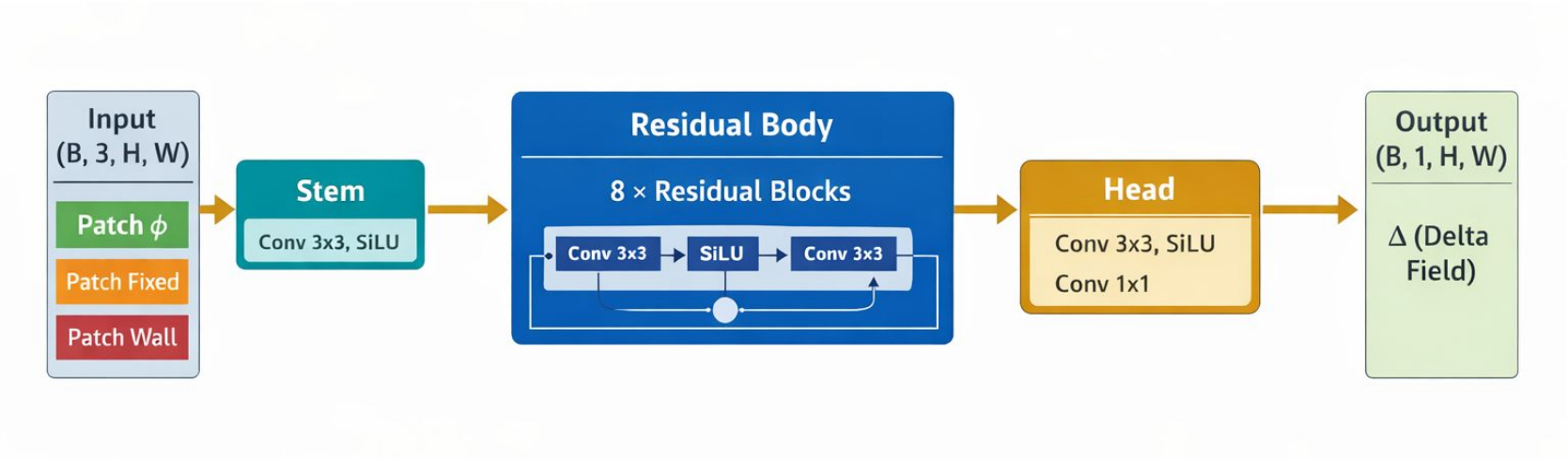
Results – Successive Over-Relaxation (SOR)



- Classical approach for dynamic object detection works as intended
- Robot “sees” the new obstacles and adjusts global path taken

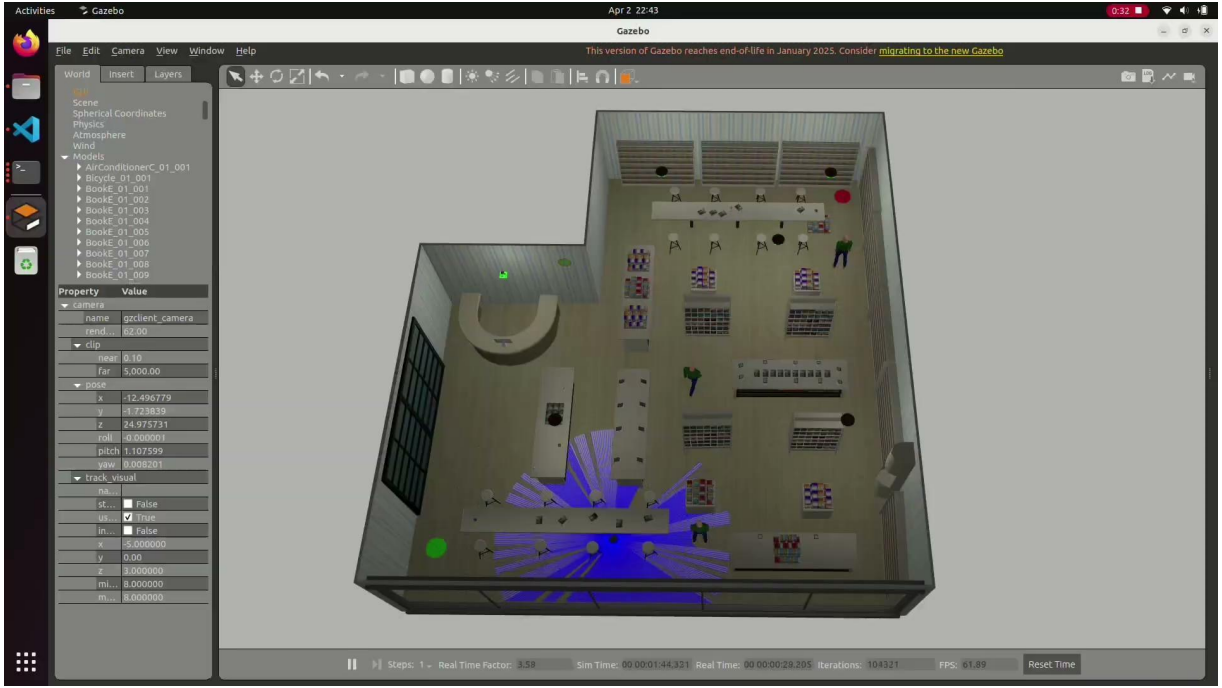
Architecture – Neural Network (NN)

- Learn perturbation field using neural network instead of explicit calculation
- Architecture motivated by ResNet-style CNN for learned PDE correction



Note: Figure generated using AI

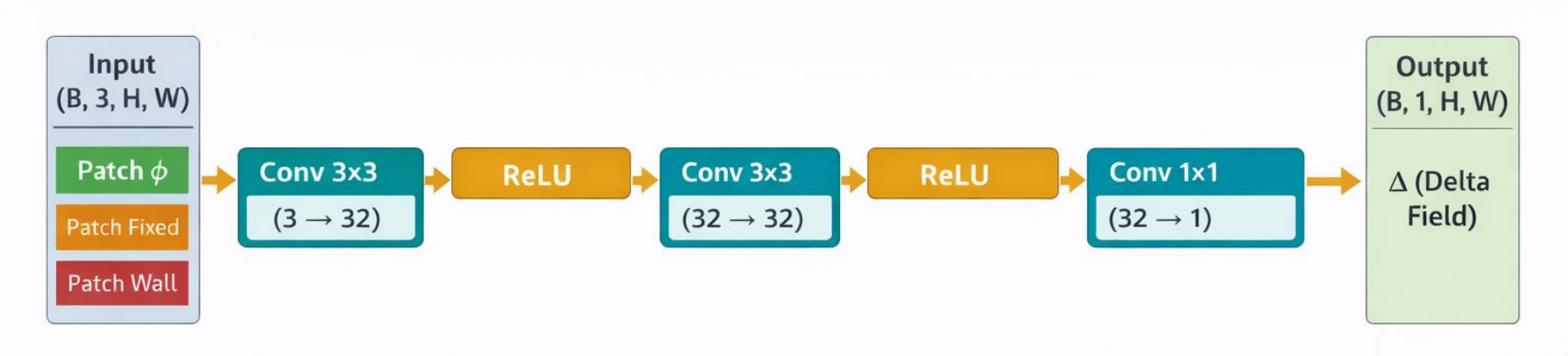
Results – Neural Network (NN)



- Method was unsuccessful, robot consistently hits the dynamic obstacle
- Pure learning approach depends heavily on NN architecture and quality

Architecture – NN Init + SOR Refinement

- NN predicted perturbation field lacks theoretical guarantees and can be incorrect
- Instead of fully trusting a NN prediction, initialize a candidate perturbation field using a simple network (fast inference), and refine it using SOR until convergence

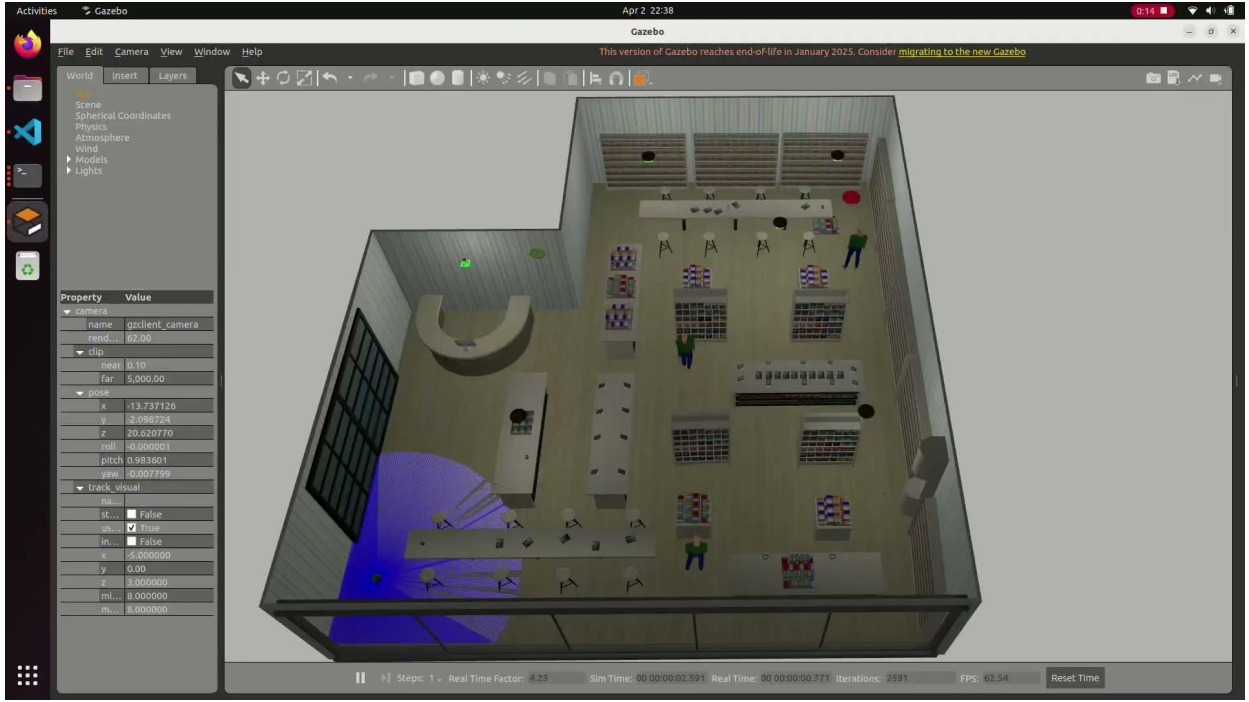


Note: Figure generated using AI

Results – NN Init + SOR Refinement

- **SOR Issues**: Runtime often takes too long to solve in dense environments
  - Average Runtime: 215 ms
  - Average Convergence: 434 iterations
  - Dense Environments: 800+ ms and over 2000+ iterations
- **NN Issues**: No guarantees of solution quality, often causes robot to collide
- **Hybrid Approach**: Initialize a candidate perturbation field from NN, then perform SOR until convergence
  - Average Runtime: 164 ms (**Speedup: 24%**)
  - Average Convergence: 317 iterations (**Speedup: 27%**)
  - Maintains theoretical guarantees of SOR
  - Solution quality lower-bounded by SOR

Results – NN Init + SOR Refinement



- Hybrid approach for dynamic object detection works as intended
- Learning is used to speed-up classical approach rather than replace it

Conclusion

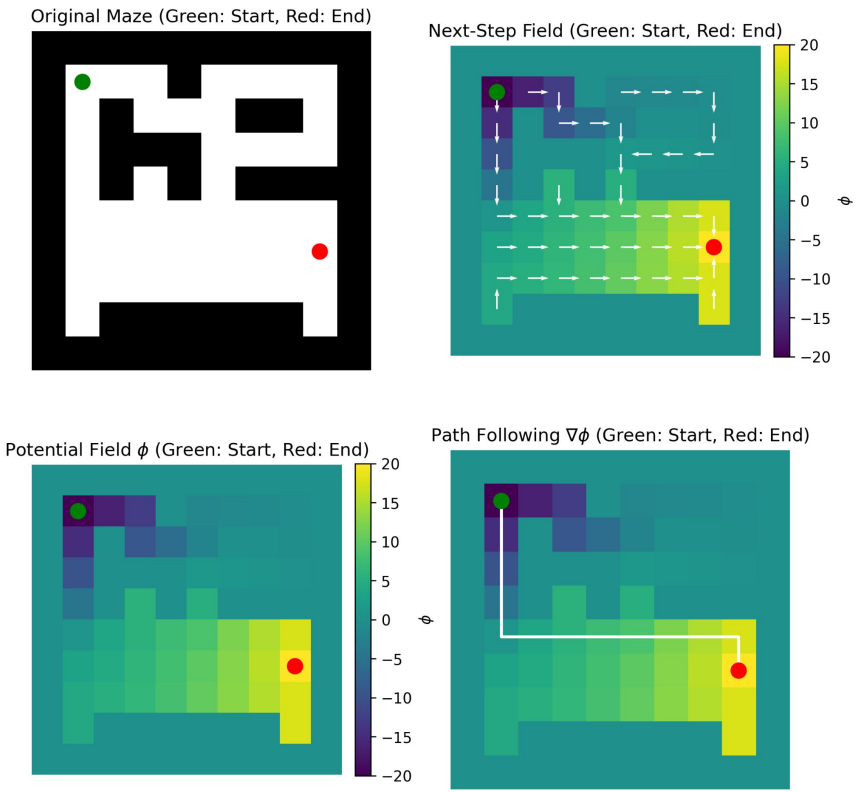
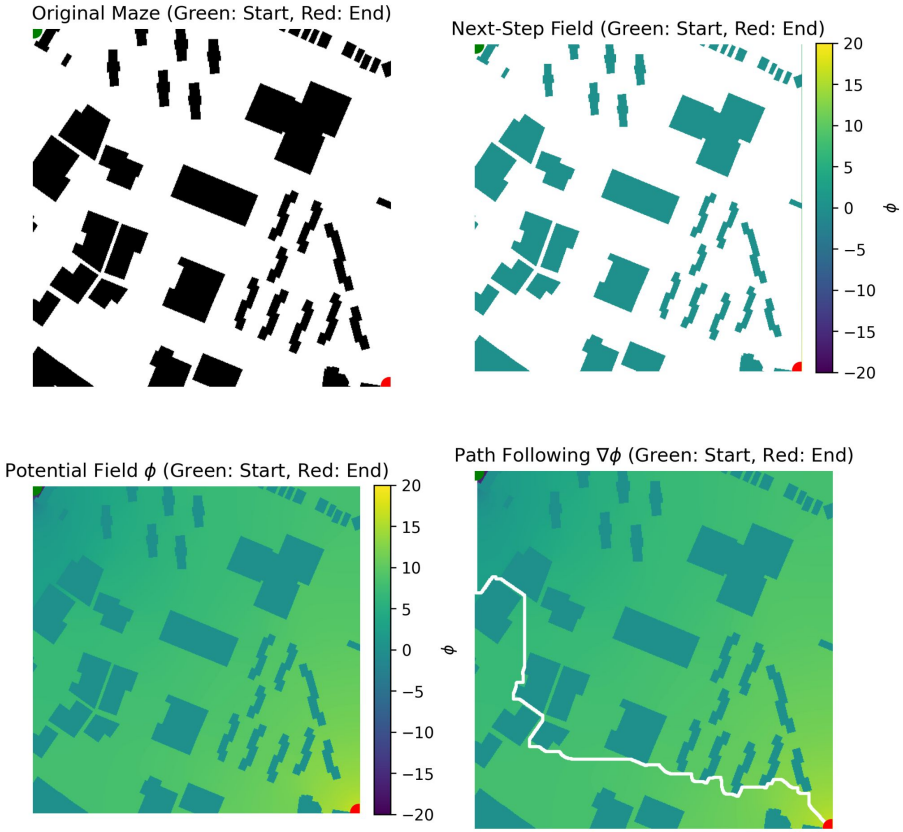
- Laplacian potential fields for global path planning with theoretical deadlock-free guarantees
- Local perturbation field estimate allows adaptation to dynamic obstacles
- **SOR**: Accurate but computationally expensive in dense environments
- **NN**: Fast but unreliable without guarantees
- **Hybrid**: Convergence speedup while preserving correctness
- **Limitations**: Discretization errors, partial observability, and real-world constraints
- **Future Work**:
  - Obstacle modelling to estimate future trajectory of obstacle from lidar scan
  - Different NN architectures for pure NN approach

Thank You!

https://github.com/williamfbx/Laplace-Pathfinder

Appendix – 2D Grid Map Toy Example

- Solves static Laplace's equation using sparse linear system Aφ = b
- Encode neighbor relationships and permissible flux flow cells in A
- Encode boundary conditions and charge sources/sinks in b
- Solve linear system using SciPy sparse linear solver
- Path following algorithm calculates local gradient at location of robot
- Take step in direction of gradient



Appendix – New York 512x512 Grid Map